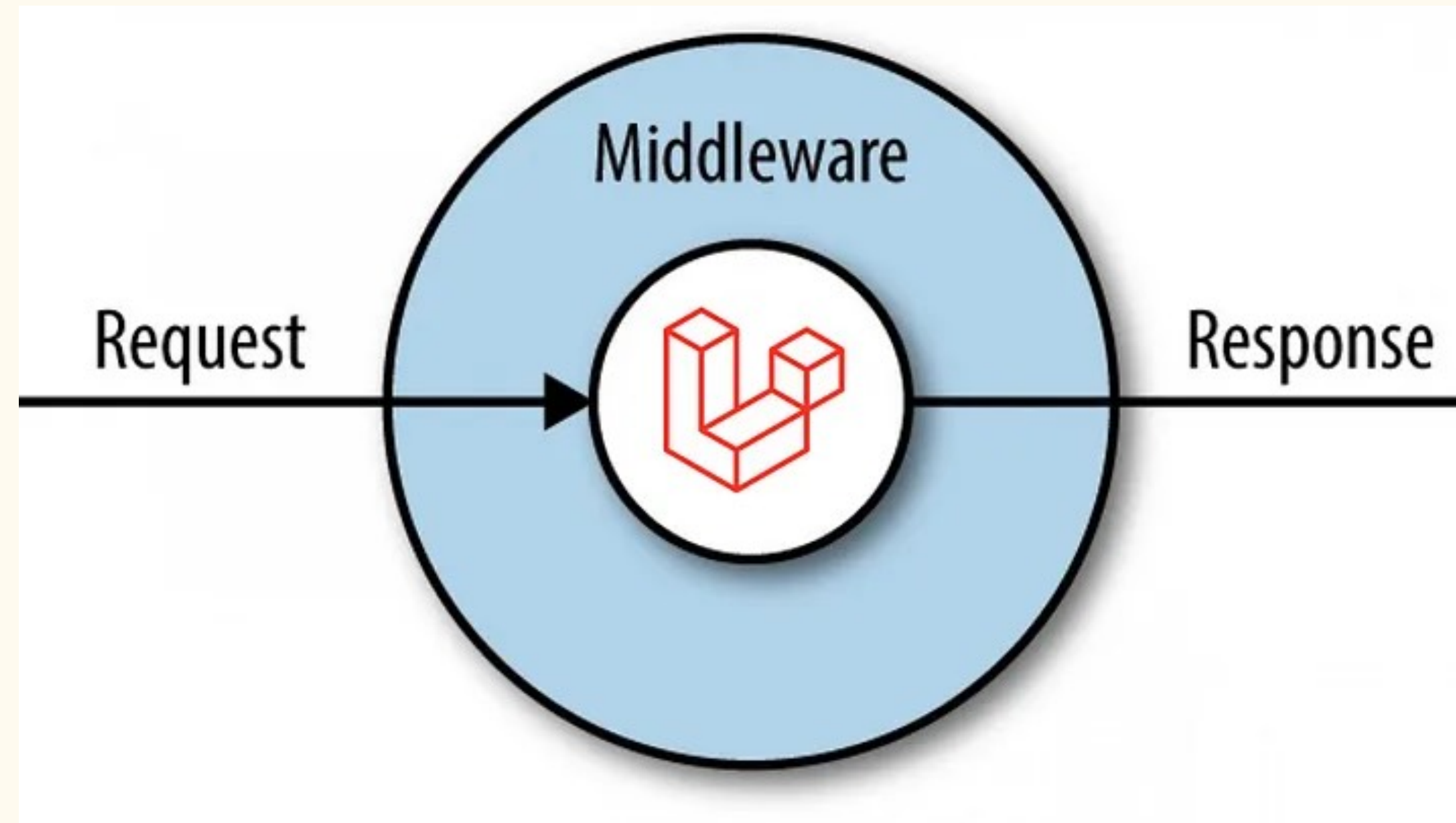
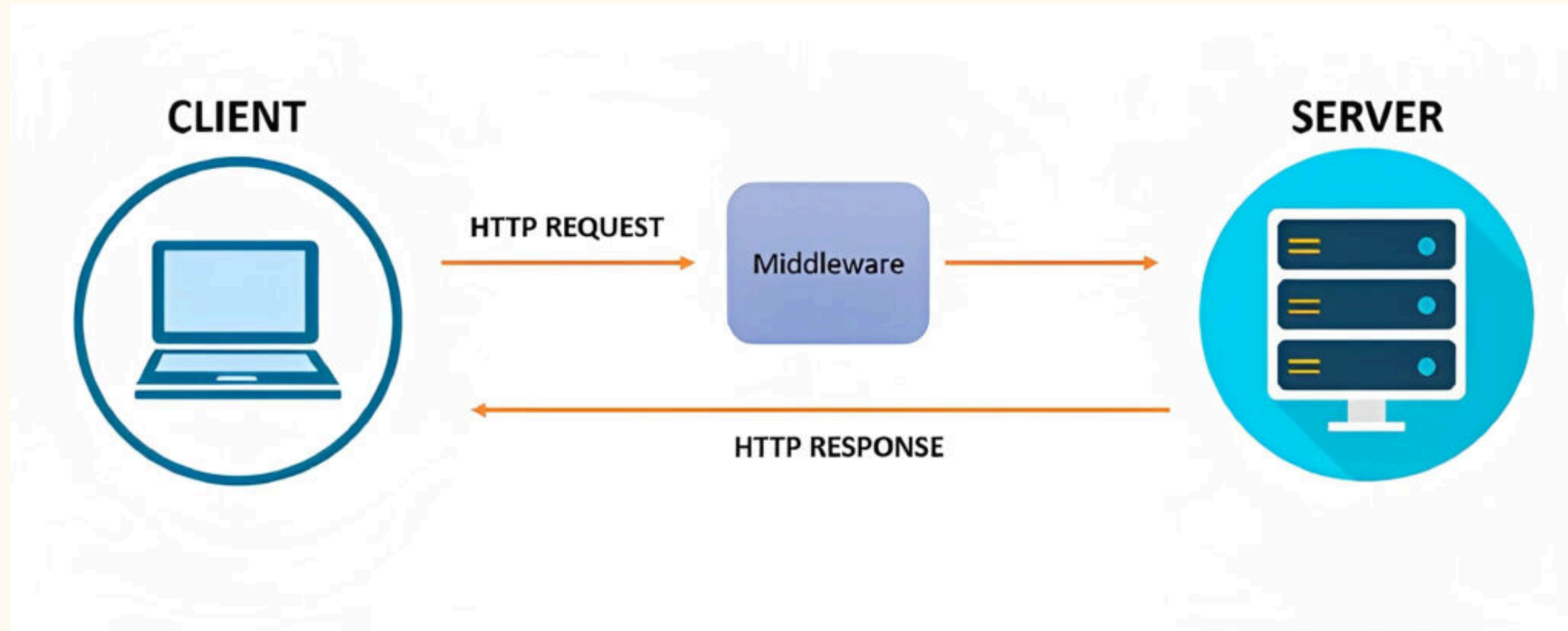
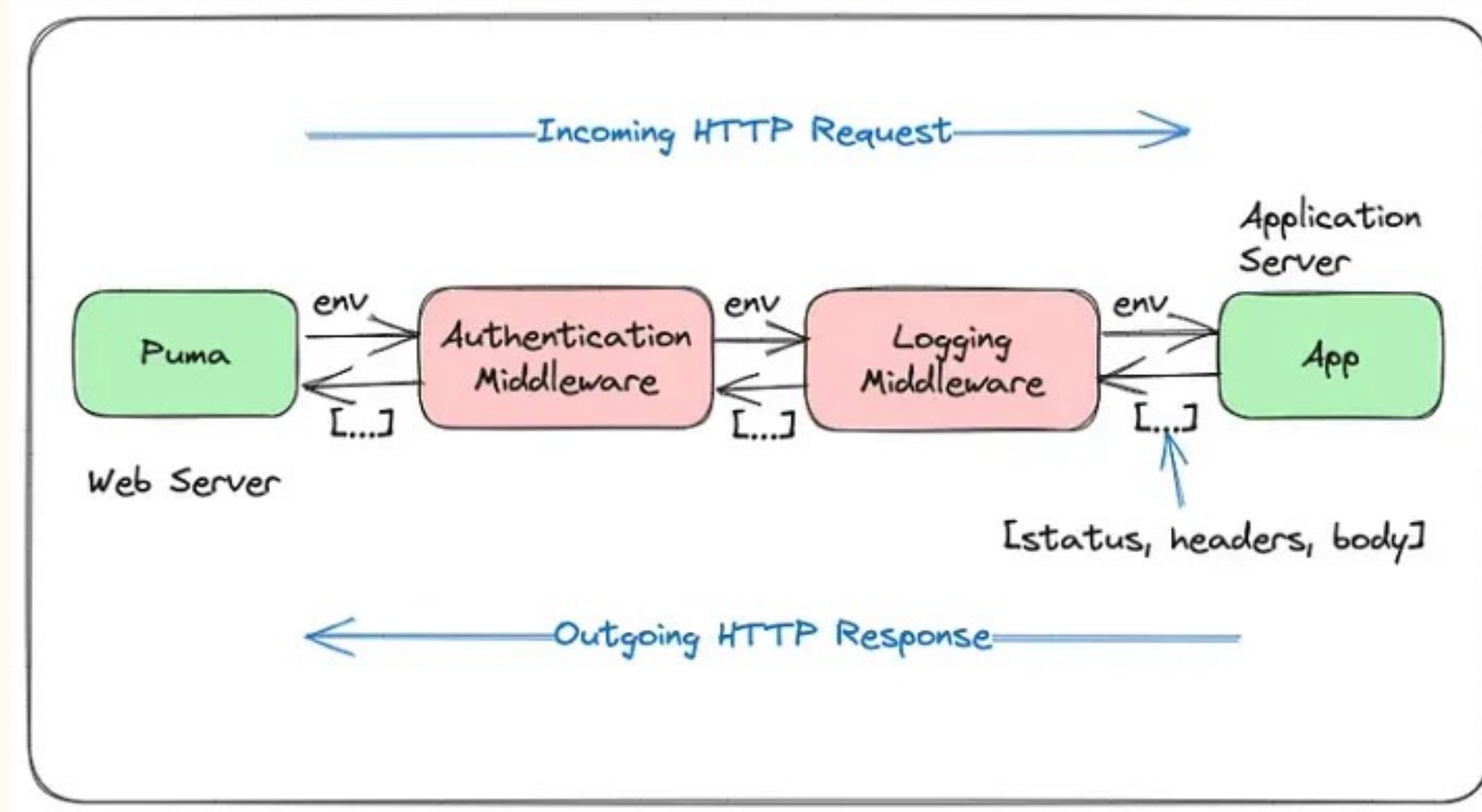


AUTH & MIDDLEWARE

I. LESSON







II. PROJECT

main.rs

```
let protected_routes: Router<AppState> = Router::new() Router<AppState>
    .merge(todos_routes()) Router<AppState>
    .merge(books_routes()) Router<AppState> // ✓ Authentication required.
    .route_layer(middleware::from_fn(required_authentication));

let app: Router = Router::new() Router<AppState>
    .merge(auth_routes()) Router<AppState>
    .merge(protected_routes) Router<AppState>
    .layer(middleware::from_fn(authenticate)) Router<AppState>
    .with_state(app_state) Router
    .layer(session_layer);
```


session.rs

```
use sqlx::PgPool;
use time::Duration;
use tower_sessions::{Expiry, SessionManagerLayer};
use tower_sessions_sqlx_store::PostgresStore;

const SESSION_EXPIRY: Duration = Duration::days(1);

1 implementation
pub struct Session;

impl Session {
    pub async fn init(pool: PgPool) -> SessionManagerLayer<PostgresStore> {
        let session_store: PostgresStore = PostgresStore::new(pool);

        session_store PostgresStore
            .migrate() impl Future<Output = Result<..., ...>>
            .await Result<(), Error>
            .expect(msg: "Failed to run session migration");

        SessionManagerLayer::new(session_store).with_expiry(Expiry::OnInactivity(SESSION_EXPIRY))
    }
}
```

Project: Simple App with Auth & Cookies

Your turn!